FPC CPP

First Person Controller / C# / Unity Asset

Technical Documentation

v1.0 Unity 2022.3+

INTRODUCTION

0. Introduction

This document covers the internal logic of the three scripts that make up the FPC CPP asset. What you will find here is the technical description of each function, the physics operations, algorithms, and math involved, so that YOU can modify or extend the "controller" with full knowledge of how it works (Not the Window Inspector to be clear, I'll make another Document about it, and I will update this same document with much, much more detail, correcting anything I've already changed in the code that no longer matches, as well as the new things that will be added that I will obviously include here. This will probably be include in the first update of this asset).

General Architecture

The asset is composed of three scripts with clearly defined responsibilities:

Script	Role	Notes
PlayerConfiguration_ConfiguracionDelJugador.cs	Pure data ScriptableObject	No logic, no MonoBehaviour inheritance. Stores all configurable parameters.
PlayerControllerSetup_ControladorDelJugadorSetup.cs	Editor-only utility (#if UNITY_EDITOR)	Registers gamepad axes in the Input Manager when the project loads.
PlayerController_ControladorDelJugador.cs	Main MonoBehaviour	All runtime mechanic logic: movement, jump, body states, camera, stamina, zoom, objects, slide, dash.

Relationship Between Scripts

PlayerConfiguration is the sole data file. PlayerController reads it every frame to make decisions but never writes to it at runtime, except during Awake, to sync heights from the scene. PlayerControllerSetup does not interact with the other two at runtime.

> Important: ScriptableObjects cannot reference objects from the scene hierarchy. That is why references to the camera, camera support, all HUD elements, and the anchor point are assigned directly in the PlayerController component Inspector.

1. PlayerConfiguration

ScriptableObject created as follows:

```
[CreateAssetMenu(fileName = "PlayerConfiguration_ConfiguracionDelJugador", menuName =
"PlayerConfiguration_ConfiguracionDelJugador", order = 0)]

public class PlayerConfiguration_ConfiguracionDelJugador : ScriptableObject
```

It does not inherit from MonoBehaviour, nor does it run Awake, Update, or any other Unity loop. It is exclusively a serialized data container that Unity persists to disk as a .asset file.

To create it: right-click in the Project window - Create - PlayerConfiguration_ConfiguracionDelJugador. Then drag it to the Configuration_Configuracion field of the PlayerController component in the Inspector.

> If Configuration_Configuracion is null when Start() runs, the controller calls enabled = false and emits a Debug.LogError. This stops all loops (Update, FixedUpdate) without propagating NullReferenceExceptions.

1.1 - Movement

Field / Variable	Type	Default	Description
AllowThePlayerToWalk	bool	true	Main permission for ApplyMovement(). If false the function returns immediately.
AllowThePlayerToRun	bool	true	Checked by ThePlayerIsRunningNow(). If false, the sprint multiplier is never applied.
AllowThePlayerToRunWhileCrouching	bool	false	Allows sprint while _currentBodyState == Crouching.
AllowThePlayerToRunWhileProne	bool	false	Allows sprint while _currentBodyState == Prone.
AllowThePlayerToMoveInTheAir	bool	true	If false, GetMovementDirection() returns _movementDirectionAtTakeoff instead of live input.
BaseSpeedOfThePlayer	float	5	Base speed in units/second. Starting point for CalculateMovementSpeed().
SpeedMultiplierWhileRunning	float	1.8	Applied as: BaseSpeed * Multiplier when ThePlayerIsRunningNow() returns true.
HoldToRun	bool	true	true = GetKey() each frame. false = input toggles _runToggleModeIsActive.

1.2 - Omni-Directional Movement

Multiplies base speed based on the movement direction and current body state. When active, CalculateMovementSpeed() computes two dot products and Lerps between the directional and lateral multiplier.

Field / Variable	Type	Default	Description
EnableOmniDirectionalMovement	bool	true	If false, CalculateMovementSpeed() returns BaseSpeedOfThePlayer without directional multipliers.
Forward / Backward / Lateral * 4 states	float	Var.	One multiplier per direction (forward/back/lateral) * body state (Standing/Crouching/Prone/InAir). 12 variables total.

1.3 - Body States

Field / Variable	Type	Default	Description
AllowThePlayerToCrouch	bool	true	Permission flag checked in ReadBodyStateInput(). If false, blocks transition to BodyState.Crouching.
HoldToCrouch	bool	false	Controls hold vs. toggle mode in ProcessKeyboardBodyStateInput().
AllowThePlayerToGoProne	bool	true	Permission equivalent for BodyState.Prone.
HoldToProne	bool	false	Hold vs. toggle for the prone posture.
EnableCooldownBetweenBodyStateTransitions	bool	true	If true, ChangeBodyState() evaluates Time.time - _timeOfLastBodyStateTransition < Cooldown.
BodyStateTransitionCooldownTime	float	0.2	Minimum seconds between voluntary transitions.
SpeedOfTheCapsuleColliderTransition	float	8	Factor t of the Lerp in UpdateCapsuleColliderTransition(): Time.deltaTime * Speed.
CapsuleColliderHeightWhileStanding	float	1.8	Overwritten in Awake() with the actual _playerCapsuleCollider.height from the scene.
CapsuleColliderHeightWhileCrouching	float	1.0	Target height of the Lerp when _currentBodyState == Crouching.
CapsuleColliderHeightWhileProne	float	0.4	Target height of the Lerp when _currentBodyState == Prone.

1.4 - Jump

Field / Variable	Type	Default	Description
AllowThePlayerToJump	bool	true	Main permission for ReadJumpInput() and ApplyJump().
HowManyJumps	int	1	Defines how many times the player can jump before needing to touch the ground.
AllowThePlayerToJumpWhileProne	bool	true	If false, ApplyJump() returns when _currentBodyState == Prone.
ForceAppliedWhenJumping	float	6	Applied with AddForce(Vector3.up * Force, ForceMode.VelocityChange). Ignores mass.
GravityMultiplierDuringTheJump	float	2	Extra gravity during ascent: Physics.gravity * (Multiplier − 1f).
AdditionalGravityMultiplierDuringTheFall	float	1.5	Additional gravity during fall: Physics.gravity * (JumpMult * FallMult - 1f).
EnableVariableJump	bool	true	Enables the variable height phase in ApplyJump().
MaximumHeldTimeOfTheJumpInput	float	0.3	Limit of _jumpInputHoldTime. Once exceeded, extra force stops.
ExtraForcePerSecondOfTheVariableJump	float	15	Applied with AddForce(Vector3.up * Force, ForceMode.Acceleration) while held.
LayersThatAreConsideredGround	LayerMask	–	Passed to Physics.CheckSphere() in GroundDetection() and to the Raycast in ProcessPhysicalSliding(). Note: CanStandUp() does not use this mask; it uses Physics.DefaultRaycastLayers with the player's own layer excluded.
RadiusOfTheGroundDetectionOverlapSphere	float	0.25	Radius of Physics.CheckSphere() in GroundDetection().
DownwardOffsetOfTheGroundDetectionOverlapSphere	float	0.05	Sphere center offset: transform.position + Vector3.down * Offset.
EnableCoyoteTime	bool	true	Allows jumping shortly after walking off a ledge.
DurationOfTheCoyoteTime	float	0.15	Window in seconds during which the late jump is accepted.
EnableJumpBuffering	bool	true	Queues a jump pressed slightly before landing.
DurationOfTheJumpBuffering	float	0.15	Seconds the buffered jump input remains valid after landing.

1.5 - Stamina

Field / Variable	Type	Default	Description
EnableTheStaminaSystem	bool	true	Master toggle. When false, no stamina is consumed or regenerated.
MaximumPlayerStamina	float	100	Maximum stamina pool.
EnableStaminaCostWhenRunning	bool	true	Whether sprinting drains stamina.
StaminaCostPerSecondWhenRunning	float	15	Stamina drained per second while sprinting.
EnableStaminaCostWhenJumping	bool	true	Whether jumping costs stamina.
StaminaCostWhenJumping	float	10	Stamina cost per jump.
EnableStaminaCostWhenCrouching	bool	false	Whether entering crouch costs stamina.
StaminaCostWhenCrouching	float	5	Stamina cost on transition to Crouching.
EnableStaminaCostWhenGoingProne	bool	false	Whether going prone costs stamina.
StaminaCostWhenGoingProne	float	5	Stamina cost on transition to Prone.
EnableStaminaCostOnEachPostureTransition	bool	false	Whether every posture change costs stamina.
StaminaCostPerPostureTransition	float	3	Cost per transition when the above is enabled.
EnableExtraStaminaCostWhenJumpingFromProne	bool	true	Applies extra stamina penalty for jumping from prone.
ExtraStaminaCostWhenJumpingFromTheProneState	float	8	Extra cost added on top of the normal jump cost.
DelayInSecondsBeforeStaminaStartsRegenerating	float	1.5	Seconds to wait after last stamina spend before regen starts.
StaminaRegenerationSpeedWhileThePlayerIsIdle	float	25	Stamina per second while standing still.
StaminaRegenerationSpeedWhileThePlayerWalks	float	10	Stamina per second while walking.
ShowTheStaminaBarOnTheHUD	bool	true	Toggles stamina bar visibility in UpdateStaminaHUD().

1.6 - Head Bob

The continuous bob displacement is calculated with two trigonometric functions. The Y axis uses sin(t) at full frequency. The X axis uses cos(t * 0.5) at half frequency and half amplitude. This produces an elliptical oscillation: for every two vertical cycles, one horizontal cycle occurs.

Field / Variable	Type	Default	Description
EnableTheHeadBobbingSystem	bool	true	If false, UpdateHeadBobbing() returns immediately.
EnableHeadBobbingWhileThePlayerWalks	bool	true	Activates the walk intensity/frequency branch.
IntensityOfHeadBobbingWhileThePlayerWalks	float	0.05	Amplitude of sin/cos while walking.
FrequencyOfHeadBobbingWhileThePlayerWalks	float	12	Timer advance speed while walking.
EnableHeadBobbingWhileThePlayerRuns	bool	true	Activates the run intensity/frequency branch.
IntensityOfHeadBobbingWhileThePlayerRuns	float	0.1	Amplitude of sin/cos while running.
FrequencyOfHeadBobbingWhileThePlayerRuns	float	16	Timer advance speed while running.
EnableReactiveHeadBobbingWhenJumpingAndLanding	bool	true	Enables impulse bob on jump and landing.
IntensityOfReactiveHeadBobbingWhenJumping	float	0.08	Vector3.back * Intensity assigned on jump.
IntensityOfReactiveHeadBobbingWhenLanding	float	0.12	Vector3.down * Intensity assigned on landing.
EnableReactiveHeadBobbingWhenCrouching	bool	true	Enables impulse when transitioning to Crouching.
IntensityOfReactiveHeadBobbingWhenCrouching	float	0.07	Vector3.down * Intensity assigned on crouch.
EnableReactiveHeadBobbingWhenGoingProne	bool	true	Enables impulse when transitioning to Prone.
IntensityOfReactiveHeadBobbingWhenGoingProne	float	0.12	Vector3.down * Intensity assigned on prone.
ReturnSpeedOfReactiveHeadBobbingToTheNeutralPosition	float	10	Factor t of Vector3.Lerp(_reactiveBobbingImpulse, Vector3.zero, Time.deltaTime * Speed) each frame.
EnableReactiveHeadBobbingWhenSliding	bool	true	Enables the head impulse when starting a slide.
IntensityOfReactiveHeadBobbingWhenSliding	float	0.15	Impulse magnitude on slide start.
EnableReactiveHeadBobbingWhenDashing	bool	true	Enables the head impulse when dashing.
IntensityOfReactiveHeadBobbingWhenDashing	float	0.1	Calculated with InverseTransformDirection, relative to dash direction.
EnableBreathingEffect	bool	true	Activates stamina-driven breathing pitch + translation sway.
BreathsPerMinuteAtRest	float	14	Breathing frequency at full stamina (range 8-40 BPM).
BreathsPerMinuteWhenExhausted	float	30	Breathing frequency at zero stamina (range 8-40 BPM).
BreathingInhaleFraction	float	0.3	Fraction of one breath cycle spent inhaling (range 0.1-0.9).
BreathingBasePitchIntensity	float	3	Camera pitch sway in degrees at full stamina.
BreathingExhaustedPitchIntensity	float	5	Camera pitch sway in degrees at zero stamina.

Field / Variable	Type	Default	Description
BreathingBaseTranslationIntensity	float	0.05	Camera Y translation at full stamina.
BreathingExhaustedTranslationIntensity	float	0.05	Camera Y translation at zero stamina.
BreathingExhaustionFadeInSpeed	float	1.5	MoveTowards step size per second at which exhaustion influence rises.
BreathingExhaustionFadeOutSpeed	float	0.8	MoveTowards step size per second at which exhaustion influence falls.

1.7 - Camera

Field / Variable	Type	Default	Description
BaseFieldOfViewOfTheCamera	float	75	FOV targeted by the Lerp when _zoomIsActive is false. Initialized in Awake().
EnableSpeedFOVEffect	bool	true	Adds a slight FOV boost when the player exceeds a speed threshold.
SpeedFOVStartPercent	float	0.8	Speed fraction of BaseSpeed at which the FOV effect begins.
SpeedFOVMaxPercent	float	1.6	Speed fraction at which the FOV effect is fully applied.
SpeedFOVMaxMultiplier	float	1.15	Maximum FOV multiplier applied by the speed effect.
SpeedFOVTransitionSpeed	float	6	Lerp speed for the speed FOV transition.
HorizontalMouseSensitivity	float	2	Multiplier of Input.GetAxis("Mouse X") or rsH * sensH.
VerticalMouseSensitivity	float	2	Multiplier of Input.GetAxis("Mouse Y") or rsV * sensV.
UpperVerticalLimitOfTheCamera	float	80	Upper clamp of _cameraAccumulatedVerticalRotationAngle.
LowerVerticalLimitOfTheCamera	float	80	Lower clamp of _cameraAccumulatedVerticalRotationAngle.
ShowTheCrosshairOnTheHUD	bool	true	Passed to CrosshairOnHUD.SetActive() in UpdateCrosshairVisibility().
HeightOfTheCameraSupportWhileStanding	float	1.6	Overwritten in Awake() with actual Y of CameraSupport.localPosition.
HeightOfTheCameraSupportWhileCrouching	float	0.7	Y target of the Lerp when _currentBodyState == Crouching.
HeightOfTheCameraSupportWhileProne	float	0.2	Y target of the Lerp when _currentBodyState == Prone.
SpeedOfTheCameraHeightTransition	float	8	Factor t of Mathf.Lerp for height: Time.deltaTime * Speed.

1.8 - Zoom

Field / Variable	Type	Default	Description
EnableTheZoomSystem	bool	true	Central permission for ReadZoomInput() and UpdateZoom().
HoldToZoom	bool	true	true = _zoomIsActive = ZoomInputHeld(). false = toggle with ZoomInputPressed().
AllowZoomWhileHoldingAnObject	bool	true	If false and _currentlyHeldObject != null, ReadZoomInput() returns.
CameraFieldOfViewDuringZoom	float	40	Target FOV of the Lerp when _zoomIsActive is true.
SpeedOfTheFieldOfViewTransitionDuringZoom	float	8	Factor t of Mathf.Lerp of FOV in UpdateZoom().
ReduceSensitivityDuringZoom	bool	true	If true, ReadCameraInput() multiplies sensH and sensV by the multiplier.
SensitivityMultiplierDuringZoom	float	0.5	sensH *= Multiplier when zoom is active.

1.9 - Object Interaction

Field / Variable	Type	Default	Description
EnableTheObjectInteractionSystem	bool	true	Key permission for ReadObjectInteractionInput().
TagOfPickableObjects	string	Recogible	Passed to hit.collider.CompareTag() in TryToPickUpTheObject().
MaximumDistanceToPickUpAnObject	float	3	Raycast length from MainCamera_CamaraPrincipal.transform.position.
DisableTheObjectColliderWhenPickingItUp	bool	true	If true, PickUpObject() disables the object's Collider.
SpeedOfTheObjectMovementTowardsTheAnchorPoint	float	15	Factor t of Vector3.Lerp in UpdateHeldObjectPosition().
HoldToRotateTheObject	bool	false	Switches between hold and toggle mode in ReadObjectInteractionInput().
RotationSpeedOfTheObjectInHand	float	90	Degrees per second in RotateObjectInHand().
TakeIntoAccountTheMassOfTheObjectWhenThrowingIt	bool	false	Switches between ForceMode.VelocityChange and ForceMode.Impulse.
MinimumObjectThrowForce	float	5	Force applied when throw is released with no charge.
MaximumObjectThrowForce	float	20	Force applied when throw is fully charged.
MaximumChargeTimeOfTheObjectThrow	float	1.5	Seconds required to reach maximum throw force.

1.10 - Advanced Movement: Slide & Dash

Slide

Field / Variable	Type	Default	Description
EnableTheSlidingSystem	bool	true	Master toggle for the slide mechanic.
UseRealPhysicsInSliding	bool	false	true = physics slide. false = arcade slide.
AllowJumpingDuringSliding	bool	true	Permits jumping while _playerIsSliding.
AllowInterruptingSliding	bool	true	Allows the player to cancel a slide early.
RecoveryTimeAfterSliding	float	0.4	Seconds of forced recovery state after slide ends.
AllowRunningImmediatelyAfterSliding	bool	false	Skips the recovery period.
EnableStaminaCostWhenSliding	bool	true	Whether sliding consumes stamina.
StaminaCostWhenSliding	float	15	Stamina consumed at slide start.
AllowSlidingOnAnySurface	bool	true	When false, restricts sliding to steeper slopes.
DurationOfSlidingInArcadeMode	float	0.8	Fixed duration of the slide in arcade mode (seconds).
InitialSpeedMultiplierOfArcadeSliding	float	2.5	Speed multiplier applied at arcade slide start.
MinimumSpeedToKeepSliding	float	1	Minimum velocity below which the arcade slide ends early.
FrictionDuringPhysicalSliding	float	0.3	Drag applied in physics slide mode. AddForce(-horizontalVel * Friction).
SlopeAccelerationMultiplierDuringSliding	float	1.5	Magnitude of AddForce(slopeDirection * Multiplier) on slopes.
ReduceCameraSensitivityDuringSliding	bool	true	If true, ReadCameraInput() applies the multiplier while sliding.
CameraSensitivityMultiplierDuringSliding	float	0.5	sensH *= Multiplier during the slide.

Dash

Field / Variable	Type	Default	Description
EnableTheDashSystem	bool	true	Master permission for ReadDashInput() and TryExecuteDash().
AllowDashInTheAir	bool	true	If false, TryExecuteDash() returns when !_playerIsOnGround.
DashForce	float	15	velocity.x = dashDir.x * Force; velocity.z = dashDir.z * Force. Direct assignment, not AddForce.
DurationOfTheDashImpulse	float	0.2	Seconds of yield return new WaitForSeconds() in ExecuteDash(). Blocks ApplyMovement().
CooldownBetweenDashUses	float	1	TryExecuteDash() returns if _timeSinceLastDash < Cooldown.
RequireDoublePressForDashOnGamepad	bool	false	Activates double-press logic in ReadDashInput() for gamepad.
MaximumTimeBetweenTheTwoPressesForDashDoublePress	float	0.3	Max window between first and second press. Exceeded = _waitingForSecondDashButtonPress = false.
AllowDashDuringSliding	bool	true	If false, TryExecuteDash() returns when sliding or in recovery.

Field / Variable	Type	Default	Description
EnableStaminaCostWhenUsingDash	bool	true	If true, TryExecuteDash() verifies and deducts stamina.
StaminaCostWhenUsingDash	float	20	If CurrentStamina < Cost, TryExecuteDash() returns without executing.

1.11 - Input: Keyboard

Field / Variable	Type	Default	Description
KeyboardKeyToMoveForward	KeyCode	W	inputVertical += 1f in GetMovementDirection().
KeyboardKeyToMoveBackward	KeyCode	S	inputVertical -= 1f.
KeyboardKeyToMoveLeft	KeyCode	A	inputHorizontal -= 1f.
KeyboardKeyToMoveRight	KeyCode	D	inputHorizontal += 1f.
KeyboardKeyToRun	KeyCode	LeftShift	GetKey() in RunInputHeld() and GetKeyDown() in ReadRunInput().
KeyboardKeyToJump	KeyCode	Space	GetKeyDown() / GetKey() / GetKeyUp() in the three jump input methods.
KeyboardKeyToCrouch	KeyCode	C	GetKeyDown() to start crouching or sliding in ProcessKeyboardBodyStateInput().
KeyboardKeyToGoProne	KeyCode	V	GetKeyDown() or GetKey() depending on hold/toggle mode.
KeyboardKeyForZoom	KeyCode	Z	Also accepts Input.GetMouseButton(1) as alternative.
KeyboardKeyToPickUpOrDropAnObject	KeyCode	E	GetKeyDown() in PickUpOrDropInputPressed().
KeyboardKeyToActivateObjectRotationInHand	KeyCode	R	GetKeyDown() or GetKey() depending on HoldToRotateTheObject.
Keyboard keys to rotate object (*4)	KeyCode	Arrows	verticalRotationInput and horizontalRotationInput in RotateObjectInHand().
MouseButtonToThrowTheObjectInHand	int	0	GetMouseButton(0) in ThrowInputHeld(), GetMouseButtonUp(0) in ThrowInputReleased().
KeyboardKeyForDash	KeyCode	Q	GetKeyDown() in DashInputPressed().

1.12 - Input: Gamepad

Field / Variable	Type	Default	Description
LeftStickDeadZone	float	0.2	if (Mathf.Abs(inputHorizontal) < DeadZone) inputHorizontal = 0f in GetMovementDirection().
RightStickDeadZone	float	0.2	Same filter for rsH and rsV in ReadCameraInput().
GamepadButtonToJump	KeyCode	JoystickButton0	Xbox: A / PS: Cross.
GamepadButtonToCrouchAndGoProne	KeyCode	JoystickButton1	Xbox: B / PS: Circle. Short press = crouch, long press = prone.
GamepadButtonToPickUpOrDropAnObject	KeyCode	JoystickButton2	Xbox: X / PS: Square.
GamepadButtonToActivateObjectRotationInHand	KeyCode	JoystickButton5	Xbox: RB / PS: R1.
GamepadButtonForDash	KeyCode	JoystickButton4	Xbox: LB / PS: L1.
GamepadButtonToRun	KeyCode	JoystickButton8	Xbox: L3 / PS: L3.
GamepadButtonForZoom	KeyCode	JoystickButton9	Also accepts _LTZoomPressedThisFrame.
GamepadButtonToThrowTheObjectInHand	KeyCode	JoystickButton10	Also accepts _RTIsActive and _RTReleasedThisFrame.
ButtonHoldTimeToBeConsideredAsProne	float	0.4	Threshold of _buttonHoldTimeForCrouchOrProneOnGamepad to interpret press as prone.

2. PlayerControllerSetup

Script wrapped in a #if UNITY_EDITOR block. Does not exist at runtime or in builds. Compiled exclusively for the Unity Editor. The file must reside inside a folder named Editor anywhere in the project.

2.1 - [InitializeOnLoad] Static Constructor

The class has the [InitializeOnLoad] attribute, which makes Unity execute the static constructor automatically when the Editor loads or recompiles:

```
[InitializeOnLoad]
public static class PlayerControllerSetup_ControladorDelJugadorSetup
{
    // ═══════════════════════════════════════════════════════════════════════════════
    #region INICIALIZACIÓN / INITIALIZATION
    // ═══════════════════════════════════════════════════════════════════════════════

    static PlayerControllerSetup_ControladorDelJugadorSetup()
    {
        RegisterGamepadAxesIfMissing();
    }

    #endregion
    // ═══════════════════════════════════════════════════════════════════════════════
```

The constructor calls RegisterGamepadAxesIfMissing(), which is also exposed as a menu item:

```
    [MenuItem("FPC - CPP / Register Gamepad Axes - Registrar Ejes del Mando")]

    public static void RegisterGamepadAxesIfMissing()
```

If the axes ever disappear from the Input Manager, they can be re-registered manually from that menu without needing to reimport the asset.

2.2 - RegisterAxis()

Before registering any axis, the function iterates the m_Axes array to verify that no axis with the same name already exists. If it does, it returns false without modifying anything:

```
        for (int i = 0; i < axesProperty.arraySize; i++)
        {
            if (axesProperty.GetArrayElementAtIndex(i).FindPropertyRelative("m_Name").stringValue == name)
                return false;
        }
```

If it does not exist, it increments arraySize, accesses the new element by index, and assigns all serialized properties. Dead zone is 0.19f, sensitivity 1f for all axes. Note: this is the dead zone stored in the Input Manager asset and applied by Unity's input system. It is distinct from LeftStickDeadZone and RightStickDeadZone (default 0.2f in the ScriptableObject), which are separate thresholds applied in code inside GetMovementDirection() and ReadCameraInput().

> The script never modifies existing axes. It only adds missing ones. There is no risk of overwriting prior project configurations.

Parameters of RegisterAxis():

Parameter	Type	Description
axesProperty	SerializedProperty	Reference to the m_Axes array in the InputManager.asset.
name	string	Axis name as it will appear in code (e.g. "RSHorizontal").
joyAxisNum	int	Unity joystick axis number (1-based; stored internally as num - 1).
inverted	bool	If true the axis is inverted. Used for vertical stick axes (RSVertical, DpadVertical).

2.3 - Registered Axes

Axis Name	Joystick Axis	Description
RSHorizontal	Axis 4	Right stick horizontal. Camera H. Read in ReadCameraInput() with Input.GetAxisRaw().
RSVertical	Axis 5	Right stick vertical. Camera V. Registered with invert = true.
DpadHorizontal	Axis 6	D-Pad H. Read in RotateObjectInHand() with Input.GetAxisRaw().
DpadVertical	Axis 7	D-Pad V. Read in RotateObjectInHand() with Input.GetAxisRaw(). Registered with invert = true.
LT	Axis 9	Analog left trigger. Read in DetectGamepadTriggers(). Activation threshold: > 0.3f.
RT	Axis 10	Analog right trigger. Read in DetectGamepadTriggers(). Threshold: > 0.3f.

3. PlayerController

Main MonoBehaviour decorated with [RequireComponent(typeof(Rigidbody))] and [RequireComponent(typeof(CapsuleCollider))]. Scene references that cannot live in the ScriptableObject are assigned in the Inspector: Configuration_Configuracion, CameraSupport_SoporteDeLaCamara, MainCamera_CamaraPrincipal, the stamina bar images, the crosshair, and the object anchor point.

3.1 - Awake()

Runs before any Start(), even if the GameObject is deactivated. Component retrieval and Rigidbody configuration:

```
_playerRigidbody = GetComponent<Rigidbody>();
_playerCapsuleCollider = GetComponent<CapsuleCollider>();
_originalCapsuleRadius = _playerCapsuleCollider.radius;

_playerRigidbody.freezeRotation = true;

_playerRigidbody.interpolation = RigidbodyInterpolation.None;

PlayerCurrentStamina = Configuration_Configuracion.MaximumPlayerStamina;
```

freezeRotation = true freezes the Rigidbody rotation on all three axes. Without this, any asymmetric ContactPoint in a collision would apply torque and the character would tilt, fall, or flip. With it active, all rotation is controlled exclusively by the script via transform.Rotate() and Quaternion.Euler().

Height synchronization from scene to ScriptableObject:

```
if (CameraSupport_SoporteDeLaCamara != null)
    _originalYPositionOfCameraSupport = CameraSupport_SoporteDeLaCamara.localPosition.y;

if (Configuration_Configuracion != null)
{
    Configuration_Configuracion.HeightOfTheCameraSupportWhileStanding = _originalYPositionOfCameraSupport;
    Configuration_Configuracion.CapsuleColliderHeightWhileStanding = _playerCapsuleCollider.height;
}
```

The scene takes precedence over the ScriptableObject: if the camera rig or CapsuleCollider were modified in the Inspector, Awake captures those values at runtime and writes them to the asset, avoiding discrepancies between the data asset and the actual scene. Awake also initializes PlayerCurrentStamina to MaximumPlayerStamina, sets _currentCameraFieldOfView to BaseFieldOfViewOfTheCamera and applies it immediately to MainCamera_CamaraPrincipal.fieldOfView, and locks and hides the cursor (Cursor.lockState = CursorLockMode.Locked, Cursor.visible = false).

To prevent the first frame from falsely triggering the landing event, both ground variables are initialized to true:

```
        _playerWasOnGroundLastFrame = true;
        _playerIsOnGround = true;
        _remainingJumps = Configuration_Configuracion.HowManyJumps;
```

Note: the dash cooldown is initialized in Start(), not in Awake(), with _timeSinceLastDash = CooldownBetweenDashUses + 1f, so the dash is available from the very first frame.

3.2 - Start()

Runs once on the first frame, after all Awake calls have completed. Validates that Configuration_Configuracion is assigned, if null, it calls enabled = false and logs an error, stopping all loops without propagating NullReferenceExceptions. On success, it calls UpdateCrosshairVisibility() and UpdateStaminaHUD() to set up the initial HUD state, saves the original stamina bar pixel width as fill reference, and initializes _timeSinceLastDash = CooldownBetweenDashUses + 1f so the dash is available from the very first frame.

3.3 - Update()

Runs every frame. Contains no physics logic and does not modify the Rigidbody. Orchestrates calls in a fixed order:

```
        if (Configuration_Configuracion == null) return;

        DetectActiveInputDevice();
        DetectGamepadTriggers();

        ReadCameraInput();
        ReadBodyStateInput();
        ReadJumpInput();
        ReadRunInput();
        ReadZoomInput();
        ReadObjectInteractionInput();
        ReadDashInput();

        if (_playerIsSliding && Configuration_Configuracion.AllowInterruptingSliding &&
_timeSinceSlidingStarted > 0f)
            _interruptSlideInputCached = _interruptSlideInputCached || InputToInterruptSliding();

        UpdateCoyoteTimeTimer();
        UpdateJumpBufferTimer();
        UpdateDashCooldownTimer();

        UpdatePostSlideRecovery();
        UpdateCapsuleColliderTransition();
        UpdateCameraHeight();
        UpdateHeadBobbing();
        UpdateZoom();
        UpdateStaminaHUD();

        if (MainCamera_CamaraPrincipal != null)
            MainCamera_CamaraPrincipal.fieldOfView = _currentCameraFieldOfView;

        EnforceCanStandUp();
        _cachedMovementDirection = GetMovementDirection();
```

GetMovementDirection() is called at the end of Update and cached in _cachedMovementDirection. FixedUpdate consumes that cached value in ApplyMovement(). This guarantees both rotation and physics use the same direction snapshot per frame, avoiding inconsistencies when Update and FixedUpdate run at different rates.

3.4 - FixedUpdate()

Physics loop at fixed rate (Time.fixedDeltaTime = 0.02s by default). Everything that assigns velocity or calls AddForce on the Rigidbody goes here. Internal order is deliberate:

```
GroundDetection();
ApplyGravity();
ApplyMovement();
ApplyJump();
ExecuteSliding();
UpdateStamina();
```

3.5 - GroundDetection()

Uses Physics.CheckSphere instead of Raycast to avoid false negatives on irregular edges or simple platform edges:

```
_playerWasOnGroundLastFrame = _playerIsOnGround;

Vector3 detectionPoint = transform.position + Vector3.down *
Configuration_Configuracion.DownwardOffsetOfTheGroundDetectionOverlapSphere;
_playerIsOnGround = Physics.CheckSphere(
    detectionPoint,
    Configuration_Configuracion.RadiusOfTheGroundDetectionOverlapSphere,
    Configuration_Configuracion.LayersThatAreConsideredGround,
    QueryTriggerInteraction.Ignore);
```

QueryTriggerInteraction.Ignore excludes Trigger Colliders from detection. Without this parameter, any trigger zone on the ground would activate _playerIsOnGround even if the player were airborne above it.

The comparison between current and previous frame state detects two edges:

- Rising edge (false - true) - calls OnLanding().
- Falling edge without jump (true - false && !_playerIsPerformingJump) - assigns _remainingCoyoteTime = DurationOfTheCoyoteTime.

OnLanding() resets _remainingJumps, clears jump state variables, cancels any active slide, assigns the reactive head bob landing impulse, and calls RestoreBodyStateOnLanding(). The latter reconstructs the correct posture, crouch/prone/standing, based on which toggle/hold state was active before the jump.

3.6 - ApplyGravity()

Unity applies Physics.gravity to all Rigidbodies with useGravity = true. This method adds extra force on top using ForceMode.Acceleration, which applies acceleration regardless of mass:

```
        if (_playerIsOnGround) return;

        if (_playerRigidbody.velocity.y < 0f)
            _playerRigidbody.AddForce(
                Physics.gravity * (Configuration_Configuracion.GravityMultiplierDuringTheJump *
Configuration_Configuracion.AdditionalGravityMultiplierDuringTheFall - 1f),
                ForceMode.Acceleration);
        else
            _playerRigidbody.AddForce(
                Physics.gravity * (Configuration_Configuracion.GravityMultiplierDuringTheJump - 1f),
                ForceMode.Acceleration);
```

The -1 in both formulas is necessary: Unity already applies Physics.gravity once. If multiplied directly by the multiplier without subtracting 1, gravity would be applied twice in the normal phase. The subtraction compensates for this.

Result: during ascent (velocity.y >= 0), the extra force added is (GravityMultiplierDuringTheJump - 1) * gravity, so the total effective gravity is GravityMultiplierDuringTheJump * gravity. During descent (velocity.y < 0), the extra force is (GravityMultiplierDuringTheJump * AdditionalGravityMultiplierDuringTheFall - 1) * gravity, making the total effective gravity GravityMultiplierDuringTheJump * AdditionalGravityMultiplierDuringTheFall * gravity.

> ForceMode.Acceleration is chosen over ForceMode.Force so the effect is independent of the Rigidbody mass. With Force, changing the character's mass would alter the jump curve.

3.7 - ApplyMovement(), GetMovementDirection(), CalculateMovementSpeed()

ApplyMovement()

Directly assigns velocity to the Rigidbody while preserving the Y component:

```
        if (!Configuration_Configuracion.AllowThePlayerToWalk) return;
        if (_playerIsSliding) return;
        if (_playerIsPerformingDash) return;

        Vector3 movementDirection = _cachedMovementDirection;
        float finalSpeed = CalculateMovementSpeed(movementDirection);

        float velocityY = (_playerIsOnGround && !_playerIsPerformingJump)
            ? Mathf.Min(_playerRigidbody.velocity.y, 0f)
            : _playerRigidbody.velocity.y;

        _playerRigidbody.velocity = new Vector3(
            movementDirection.x * finalSpeed,
            velocityY,
            movementDirection.z * finalSpeed);

        if (_playerIsOnGround)
            _movementDirectionAtTakeoff = movementDirection;
```

Directly assigning .x and .z (rather than AddForce) makes movement immediate: every frame the horizontal velocity exactly mirrors the input, with no accumulated inertia. Releasing the input stops movement immediately, the expected behaviour for an FPS for example.

> AddForce would accumulate velocity frame by frame. With direct assignment, releasing the input stops movement immediately.

GetMovementDirection()

Builds the direction vector in the player's local space:

```
if (!_playerIsOnGround && !Configuration_Configuracion.AllowThePlayerToMoveInTheAir)
    return _movementDirectionAtTakeoff;

float inputHorizontal = 0f;
float inputVertical = 0f;

if (_isUsingGamepad)
{
    inputHorizontal = Input.GetAxisRaw("Horizontal");
    inputVertical = Input.GetAxisRaw("Vertical");
    if (Mathf.Abs(inputHorizontal) < Configuration_Configuracion.LeftStickDeadZone) inputHorizontal = 0f;
    if (Mathf.Abs(inputVertical) < Configuration_Configuracion.LeftStickDeadZone) inputVertical = 0f;
}
else
{
    if (Input.GetKey(Configuration_Configuracion.KeyboardKeyToMoveForward)) inputVertical += 1f;
    if (Input.GetKey(Configuration_Configuracion.KeyboardKeyToMoveBackward)) inputVertical -= 1f;
    if (Input.GetKey(Configuration_Configuracion.KeyboardKeyToMoveRight)) inputHorizontal += 1f;
    if (Input.GetKey(Configuration_Configuracion.KeyboardKeyToMoveLeft)) inputHorizontal -= 1f;
}

if (inputHorizontal == 0f && inputVertical == 0f) return Vector3.zero;

Vector3 direction = transform.right * inputHorizontal + transform.forward * inputVertical;
direction.y = 0f;
return direction.normalized;
```

direction.y = 0f is critical. Without it, if the camera points up or down, the forward component would have a non-zero Y and the character would move diagonally relative to the ground.

GetAxisRaw() returns the axis value without Unity's internal smoothing applied by GetAxis(), eliminating the perceptible lag when starting and stopping movement.

CalculateMovementSpeed()

Calculates the final speed applying the omnidirectionality multipliers:

```
        float baseSpeed = Configuration_Configuracion.BaseSpeedOfThePlayer * (playerIsRunning ?
Configuration_Configuracion.SpeedMultiplierWhileRunning : 1f);

        if (!Configuration_Configuracion.EnableOmniDirectionalMovement) return baseSpeed;

        float dotForward = Vector3.Dot(transform.forward, movementDirection);
        float dotLateral = Mathf.Abs(Vector3.Dot(transform.right, movementDirection));

        float directionalMultiplier;
        float lateralMultiplier;

        switch (_currentBodyState)
        {
            case BodyState.Crouching:
                directionalMultiplier = dotForward >= 0f ?
Configuration_Configuracion.ForwardOmnidirectionalMultiplierWhileCrouching :
Configuration_Configuracion.BackwardOmnidirectionalMultiplierWhileCrouching;
                lateralMultiplier =
Configuration_Configuracion.LateralOmnidirectionalMultiplierWhileCrouching;
                break;
            case BodyState.Prone:
                directionalMultiplier = dotForward >= 0f ?
Configuration_Configuracion.ForwardOmnidirectionalMultiplierWhileProne :
Configuration_Configuracion.BackwardOmnidirectionalMultiplierWhileProne;
                lateralMultiplier = Configuration_Configuracion.LateralOmnidirectionalMultiplierWhileProne;
                break;
            case BodyState.InTheAir:
                directionalMultiplier = dotForward >= 0f ?
Configuration_Configuracion.ForwardOmnidirectionalMultiplierWhileInTheAir :
Configuration_Configuracion.BackwardOmnidirectionalMultiplierWhileInTheAir;
                lateralMultiplier =
Configuration_Configuracion.LateralOmnidirectionalMultiplierWhileInTheAir;
                break;
            default:
                directionalMultiplier = dotForward >= 0f ?
Configuration_Configuracion.ForwardOmnidirectionalMultiplierWhileStanding :
Configuration_Configuracion.BackwardOmnidirectionalMultiplierWhileStanding;
                lateralMultiplier =
Configuration_Configuracion.LateralOmnidirectionalMultiplierWhileStanding;
                break;
        }

        return baseSpeed * Mathf.Lerp(directionalMultiplier, lateralMultiplier, dotLateral);
```

The Lerp uses dotLateral as t. When movement is straight (dotLateral ≈ 0), it returns the directional multiplier. When completely lateral (dotLateral ≈ 1), it returns the lateral multiplier. Diagonal movement produces a proportional intermediate value.

3.8 - Body States

The private enum BodyState has 4 values: Standing, Crouching, Prone, InTheAir. Active state lives in _currentBodyState.

Keyboard and Gamepad Differences

On keyboard, crouching and going prone have separate KeyCodes. On gamepad, the same button (JoystickButton1) handles both postures. The distinction is made by accumulating _buttonHoldTimeForCrouchOrProneOnGamepad with Time.deltaTime each frame the button is held:

```
        if (buttonHeld)
            _buttonHoldTimeForCrouchOrProneOnGamepad += Time.deltaTime;


        if (buttonHeld && _buttonHoldTimeForCrouchOrProneOnGamepad >=
Configuration_Configuracion.ButtonHoldTimeToBeConsideredAsProne && !_gamepadButtonAlreadyInterpretedAsProne)
        {
            _gamepadButtonAlreadyInterpretedAsProne = true;


        if (buttonReleased)
        {
            if (!_gamepadButtonAlreadyInterpretedAsProne)
            {

                if (ThePlayerIsRunningNow() && Configuration_Configuracion.EnableTheSlidingSystem &&
_playerIsOnGround)
                {
                    StartSliding();
                }
                else - Process as Crouch


            _buttonHoldTimeForCrouchOrProneOnGamepad = 0f;
            _gamepadButtonAlreadyInterpretedAsProne = false;
```

If both postures are activated in the same frame (possible on keyboard with separate keys), the most recent one wins using the timestamps _timestampOfLastCrouchInputPulse and _timestampOfLastProneInputPulse.

CanStandUp()

```
        float difference = Configuration_Configuracion.CapsuleColliderHeightWhileStanding -
_playerCapsuleCollider.height;
        if (difference <= 0.05f) return true;

        int maskWithoutPlayer = Physics.DefaultRaycastLayers & ~(1 << gameObject.layer);

        bool result = !Physics.SphereCast(
            transform.position + Vector3.up * (_playerCapsuleCollider.center.y),
            _playerCapsuleCollider.radius,
            Vector3.up,
            out RaycastHit hit,
            Configuration_Configuracion.CapsuleColliderHeightWhileStanding - _playerCapsuleCollider.radius *
2f,
            maskWithoutPlayer,
            QueryTriggerInteraction.Ignore);
```

The SphereCast starts from the capsule's center (center.y) with the collider's own radius, sweeping upward a distance of CapsuleColliderHeightWhileStanding - radius * 2f. The mask is Physics.DefaultRaycastLayers with the player's own layer excluded, so the collider does not collide with itself. If geometry is detected, the player cannot stand up. The 0.05f early-return threshold prevents false negatives when the collider is already at or near standing height.

ChangeBodyState(BodyState newState, bool voluntaryChange)

The voluntaryChange parameter determines whether the cooldown and stamina costs are checked. If false (landing, forced sliding, jump-triggered state change), both checks are bypassed. If true (player input), the cooldown is enforced and stamina is deducted if applicable.

3.9 - UpdateCapsuleColliderTransition()

```
        float targetHeight = _currentBodyState switch
        {
            BodyState.Crouching => Configuration_Configuracion.CapsuleColliderHeightWhileCrouching,
            BodyState.Prone => Configuration_Configuracion.CapsuleColliderHeightWhileProne,
            _ => Configuration_Configuracion.CapsuleColliderHeightWhileStanding
        };

        float targetRadius = _currentBodyState == BodyState.Prone
            ? targetHeight * 0.5f
            : _originalCapsuleRadius;

        if (Mathf.Abs(_playerCapsuleCollider.height - targetHeight) < 0.001f)
        {
            _playerCapsuleCollider.height = targetHeight;
            _playerCapsuleCollider.radius = targetRadius;
            _playerCapsuleCollider.center = new Vector3(0f, targetHeight * 0.5f, 0f);
            return;
        }

        _playerCapsuleCollider.radius = Mathf.Lerp(_playerCapsuleCollider.radius, targetRadius,
Time.deltaTime * Configuration_Configuracion.SpeedOfTheCapsuleColliderTransition);
        _playerCapsuleCollider.height = Mathf.Lerp(_playerCapsuleCollider.height, targetHeight,
Time.deltaTime * Configuration_Configuracion.SpeedOfTheCapsuleColliderTransition);
        _playerCapsuleCollider.center = new Vector3(0f, _playerCapsuleCollider.height * 0.5f, 0f);
```

The center is always recalculated as height * 0.5f. The CapsuleCollider is anchored at its geometric center: if the center is not moved when the height changes, the collider grows upward and downward simultaneously, causing the player to float or sink. In the Prone state, the collider radius is also interpolated toward targetHeight * 0.5f (half the prone height) to match the flattened capsule shape; in all other states it returns to the original radius captured in Awake().

The 0.001f threshold stops the Lerp. Without it, the asymptotic Lerp would never reach the exact target and the collider would oscillate indefinitely.

> StartSliding() assigns the collider height instantly without Lerp: _playerCapsuleCollider.height = crouchHeight. If the Lerp took frames to reduce the collider, the player would collide with low geometry they are trying to pass through.

3.10 - Jump: ReadJumpInput(), ApplyJump(), ExecuteJump()

Jump Buffer

When detecting JumpInputPressed() in Update:

```
        if (JumpInputPressed())
            _remainingJumpBufferTime = Configuration_Configuracion.EnableJumpBuffering ?
Configuration_Configuracion.DurationOfTheJumpBuffering : 0f;
```

The timer decrements by Time.deltaTime each frame. ApplyJump() in FixedUpdate checks _remainingJumpBufferTime > 0f as a condition. If the player presses jump before landing and the buffer has not expired, the jump executes upon touching the ground.

Coyote Time

When the falling edge (leaving the ground without jumping) is detected in GroundDetection():

```
        if (!_playerIsOnGround && _playerWasOnGroundLastFrame && !_playerIsPerformingJump)
            _remainingCoyoteTime = Configuration_Configuracion.EnableCoyoteTime ?
Configuration_Configuracion.DurationOfTheCoyoteTime : 0f;
```

ApplyJump() accepts the jump if _playerIsOnGround || (_remainingCoyoteTime > 0f && EnableCoyoteTime). During the Coyote Time window, the player can jump even though they are technically in the air.

ExecuteJump()

```
        Vector3 currentVelocity = _playerRigidbody.velocity;
        currentVelocity.y = 0f;
        _playerRigidbody.velocity = currentVelocity;
        _playerRigidbody.AddForce(Vector3.up * Configuration_Configuracion.ForceAppliedWhenJumping,
ForceMode.VelocityChange);

        _remainingJumps--;
        _playerIsPerformingJump = true;
        _jumpInputHoldTime = 0f;
        _remainingJumpBufferTime = 0f;
        _remainingCoyoteTime = 0f;

        _lastBodyStateWhileOnGround = _currentBodyState;
        ChangeBodyState(BodyState.InTheAir, false);
```

The velocity.y reset before AddForce is necessary for consistency. If the Jump Buffer activates the jump exactly upon landing, velocity.y could be negative at that instant. Without the reset, the force would be applied on top of that negative velocity, producing a lower-than-expected jump. With the reset, all jumps start from velocity.y = 0f and produce the same height.

ForceMode.VelocityChange applies velocity directly without dividing by mass. ForceMode.Impulse would produce different results depending on the Rigidbody mass.

Variable Jump

While _playerIsPerformingJump is true, the jump input is held, and _jumpInputHoldTime < MaximumHeldTimeOfTheJumpInput, this is applied every FixedUpdate tick:

```
        if (_playerIsPerformingJump && Configuration_Configuracion.EnableVariableJump && JumpInputHeld())
        {
            if (_jumpInputHoldTime < Configuration_Configuracion.MaximumHeldTimeOfTheJumpInput)
            {
                _playerRigidbody.AddForce(Vector3.up *
Configuration_Configuracion.ExtraForcePerSecondOfTheVariableJump, ForceMode.Acceleration);
                _jumpInputHoldTime += Time.fixedDeltaTime;
```

```
            }
        }
```

ForceMode.Acceleration is correct here: it is a continuous force per second, independent of mass. When the button is released, ReadJumpInput() detects JumpInputReleased() and forces _jumpInputHoldTime = MaximumHeldTimeOfTheJumpInput + 1f, cutting the extra phase in the next tick.

TransitionToStandingAndJump() - coroutine

Jumping from the prone state requires waiting for the CapsuleCollider to reach standing height before applying the force:

```
        ChangeBodyState(BodyState.Standing, false);
        float waitedTime = 0f;

        while (Mathf.Abs(_playerCapsuleCollider.height -
Configuration_Configuracion.CapsuleColliderHeightWhileStanding) > 0.15f && waitedTime < 0.5f)
        {
            waitedTime += Time.deltaTime;
            yield return null;
        }

        ExecuteJump(fromProneState: true);
        _systemWaitingForTransitionToPerformJump = false;
```

The 0.5f timeout prevents an infinite loop if a ceiling blocks the transition. The reference _proneJumpCoroutineActive allows cancelling the coroutine with StopCoroutine() if the situation changes before it completes.

3.11 - UpdateStamina() and SpendStamina()

UpdateStamina() runs in FixedUpdate. Horizontal movement detection uses squared magnitude to avoid an unnecessary Mathf.Sqrt():

```
        bool playerIsMoving = (_playerRigidbody.velocity.x * _playerRigidbody.velocity.x +
_playerRigidbody.velocity.z * _playerRigidbody.velocity.z) > 0.01f;
```

If the player is running and the cost is active, it calls SpendStamina(Cost * Time.fixedDeltaTime) and returns. In any other case, it accumulates _timeElapsedWithoutSpendingStamina. When it exceeds the configured delay, it regenerates:

```
            PlayerCurrentStamina = Mathf.Min(PlayerCurrentStamina + regenerationRate * Time.fixedDeltaTime,
Configuration_Configuracion.MaximumPlayerStamina);
```

SpendStamina() is the sole write point for consumption in the entire system:

```
    private void SpendStamina(float cantidad)
    {
        PlayerCurrentStamina = Mathf.Max(PlayerCurrentStamina - cantidad, 0f);
        _timeElapsedWithoutSpendingStamina = 0f;
    }
```

The timer reset guarantees the regeneration delay always restarts from zero on any expenditure, regardless of which subsystem calls it (run, jump, slide, dash, posture).

3.12 - UpdateStaminaHUD()

The bar receives two parallel updates each frame, fillAmount and rectTransform width, both via Lerp at speed 12f:

```
                float proportion = PlayerCurrentStamina / Configuration_Configuracion.MaximumPlayerStamina;
                CurrentStaminaBarImage_ImagenDeLaBarraDeResistenciaActual.fillAmount =
                    Mathf.Lerp(CurrentStaminaBarImage_ImagenDeLaBarraDeResistenciaActual.fillAmount,
proportion, Time.deltaTime * 12f);

                if (_originalWidthInPixelsOfStaminaBar > 0f)
                {
                    Vector2 size =
CurrentStaminaBarImage_ImagenDeLaBarraDeResistenciaActual.rectTransform.sizeDelta;
                    size.x = Mathf.Lerp(size.x, _originalWidthInPixelsOfStaminaBar * proportion,
Time.deltaTime * 12f);
                    CurrentStaminaBarImage_ImagenDeLaBarraDeResistenciaActual.rectTransform.sizeDelta = size;
                }
```

_originalWidthInPixelsOfStaminaBar is saved in Start() with rectTransform.sizeDelta.x and is the base reference so the width is always proportional to the correct value, regardless of whether the rectTransform was resized in the Editor.

3.13 - ReadCameraInput()

Horizontal rotation rotates the full player body. Vertical rotation rotates only the CameraSupport. Both sensitivity values are first multiplied by the active modifiers (zoom, slide) before being applied:

```
        float sensitivityH = Configuration_Configuracion.HorizontalMouseSensitivity;
        float sensitivityV = Configuration_Configuracion.VerticalMouseSensitivity;

        if (_zoomIsActive && Configuration_Configuracion.ReduceSensitivityDuringZoom)
        {
            sensitivityH *= Configuration_Configuracion.SensitivityMultiplierDuringZoom;
            sensitivityV *= Configuration_Configuracion.SensitivityMultiplierDuringZoom;
        }

        if (_playerIsSliding && Configuration_Configuracion.ReduceCameraSensitivityDuringSliding)
        {
            sensitivityH *= Configuration_Configuracion.CameraSensitivityMultiplierDuringSliding;
            sensitivityV *= Configuration_Configuracion.CameraSensitivityMultiplierDuringSliding;
        }

        transform.Rotate(Vector3.up * inputX);

        _cameraAccumulatedVerticalRotationAngle -= inputY;
        _cameraAccumulatedVerticalRotationAngle = Mathf.Clamp(
            _cameraAccumulatedVerticalRotationAngle,
            -Configuration_Configuracion.UpperVerticalLimitOfTheCamera,
            Configuration_Configuracion.LowerVerticalLimitOfTheCamera);

        CameraSupport_SoporteDeLaCamara.localRotation =
Quaternion.Euler(_cameraAccumulatedVerticalRotationAngle + _breathingPitchOffset, 0f, 0f);
```

The vertical angle is accumulated in a float rather than reading the current rig rotation, because Quaternion rotations cannot be summed directly and predictably. The float accumulator allows Mathf.Clamp to be applied correctly and Quaternion.Euler to be assigned each frame.

The RSHorizontal and RSVertical axes are read inside empty try/catch blocks. If PlayerControllerSetup has not run yet (e.g. first project import), Input.GetAxisRaw() with an unregistered axis throws an ArgumentException. The silent try/catch leaves the values at 0f without crashing.

3.14 - UpdateCameraHeight()

The Y position of CameraSupport is the sum of three independent components: the base height (Lerped by body state), the continuous bob offset, and the reactive bob impulse. To Lerp only the base without being affected by the bob offsets, the clean base Y is first extracted and then recomposed:

```
        Vector3 position = CameraSupport_SoporteDeLaCamara.localPosition;

        float yBase = Mathf.Lerp(
            position.y - _continuousBobbingOffset.y - _reactiveBobbingImpulse.y -
_breathingTranslationOffset,
            targetHeight,
            Time.deltaTime * Configuration_Configuracion.SpeedOfTheCameraHeightTransition);

        position.x = _continuousBobbingOffset.x + _reactiveBobbingImpulse.x;
        position.y = yBase + _continuousBobbingOffset.y + _reactiveBobbingImpulse.y +
_breathingTranslationOffset;
        position.z = _continuousBobbingOffset.z + _reactiveBobbingImpulse.z;

        CameraSupport_SoporteDeLaCamara.localPosition = position;
```

3.15 - UpdateHeadBobbing()

Horizontal speed is calculated with a manual magnitude to exclude the vertical component, same as in the stamina movement detection:

```
        float horizontalSpeed = Mathf.Sqrt(
            _playerRigidbody.velocity.x * _playerRigidbody.velocity.x +
            _playerRigidbody.velocity.z * _playerRigidbody.velocity.z);
```

The continuous bob uses two sine functions at different frequencies:

```
        if (playerIsMovingOnGround)
        {
            float baseIntensity;
            float baseFrequency;

            if (playerIsRunningNow && Configuration_Configuracion.EnableHeadBobbingWhileThePlayerRuns)
            {
                baseIntensity = Configuration_Configuracion.IntensityOfHeadBobbingWhileThePlayerRuns;
                baseFrequency = Configuration_Configuracion.FrequencyOfHeadBobbingWhileThePlayerRuns;
            }
            else if (Configuration_Configuracion.EnableHeadBobbingWhileThePlayerWalks)
            {
                baseIntensity = Configuration_Configuracion.IntensityOfHeadBobbingWhileThePlayerWalks;
                baseFrequency = Configuration_Configuracion.FrequencyOfHeadBobbingWhileThePlayerWalks;
            }
            else
            {
                baseIntensity = 0f;
                baseFrequency = Configuration_Configuracion.FrequencyOfHeadBobbingWhileThePlayerWalks;
            }

            // —— Multiplicador por postura / Posture multiplier ——————————————
            if (_currentBodyState == BodyState.Crouching)
            {
                baseIntensity *= 0.75f;
                baseFrequency *= 0.75f;
            }
            else if (_currentBodyState == BodyState.Prone)
            {
                baseIntensity *= 0.3f;
                baseFrequency *= 0.6f;
            }

            _continuousBobbingTimer += Time.deltaTime * baseFrequency;
            _continuousBobbingOffset.y = Mathf.Sin(_continuousBobbingTimer) * baseIntensity;
            _continuousBobbingOffset.x = Mathf.Cos(_continuousBobbingTimer * 0.5f) * baseIntensity * 0.5f;
        }
        else
        {
            _continuousBobbingTimer = 0f;
```

```
            _continuousBobbingOffset = Vector3.Lerp(
                _continuousBobbingOffset,
                Vector3.zero,
                Time.deltaTime *
Configuration_Configuracion.ReturnSpeedOfReactiveHeadBobbingToTheNeutralPosition);
        }

        _breathingTranslationOffset = Mathf.Lerp(_breathingTranslationOffset, breathingTranslationTarget,
Time.deltaTime * 3f);

        _reactiveBobbingImpulse = Vector3.Lerp(
            _reactiveBobbingImpulse,
            Vector3.zero,
            Time.deltaTime *
Configuration_Configuracion.ReturnSpeedOfReactiveHeadBobbingToTheNeutralPosition);
```

This produces an elliptical oscillation: for every two vertical cycles, one horizontal cycle occurs. The result is a natural figure-eight head movement rather than a simple vertical bounce.

The breathing system uses a stamina-driven exhaustion influence value:

```
        float breathingPitchTarget = 0f;
        float breathingTranslationTarget = 0f;

        if (Configuration_Configuracion.EnableBreathingEffect)
        {
            float rawExhaustion = Configuration_Configuracion.EnableTheStaminaSystem
                ? 1f - (PlayerCurrentStamina / Configuration_Configuracion.MaximumPlayerStamina)
                : 0f;

            float fadeSpeed = rawExhaustion > _smoothedExhaustionInfluence
                ? Configuration_Configuracion.BreathingExhaustionFadeInSpeed
                : Configuration_Configuracion.BreathingExhaustionFadeOutSpeed;
            _smoothedExhaustionInfluence = Mathf.MoveTowards(_smoothedExhaustionInfluence, rawExhaustion,
fadeSpeed * Time.deltaTime);

            float bpm = Mathf.Lerp(
                Configuration_Configuracion.BreathsPerMinuteAtRest,
                Configuration_Configuracion.BreathsPerMinuteWhenExhausted,
                _smoothedExhaustionInfluence);

            _breathingTimer += Time.deltaTime * (bpm / 60f);
            _breathingTimer %= 1f;

            float inhaleFraction = Configuration_Configuracion.BreathingInhaleFraction;
            float sineInput = _breathingTimer < inhaleFraction
                ? (_breathingTimer / inhaleFraction) * Mathf.PI
                : Mathf.PI + ((_breathingTimer - inhaleFraction) / (1f - inhaleFraction)) * Mathf.PI;

            float wave = Mathf.Sin(sineInput);

            float pitchIntensity = Configuration_Configuracion.BreathingBasePitchIntensity
                + Configuration_Configuracion.BreathingExhaustedPitchIntensity *
_smoothedExhaustionInfluence;

            float translationIntensity = Configuration_Configuracion.BreathingBaseTranslationIntensity
                + Configuration_Configuracion.BreathingExhaustedTranslationIntensity *
_smoothedExhaustionInfluence;

            breathingPitchTarget = -wave * pitchIntensity;
            breathingTranslationTarget = wave * translationIntensity;
        }

        _breathingPitchOffset = Mathf.Lerp(_breathingPitchOffset, breathingPitchTarget, Time.deltaTime * 3f);
```

The breathing pitch offset is applied to the CameraSupport vertical rotation in UpdateCameraHeight(), and the translation offset is added to the Y component alongside the bob offsets. This separates breathing from the head-bob cycle so both can run independently.

Reactive impulses are damped each frame:

```
        _reactiveBobbingImpulse = Vector3.Lerp(
```

```
            _reactiveBobbingImpulse,
            Vector3.zero,
            Time.deltaTime *
Configuration_Configuracion.ReturnSpeedOfReactiveHeadBobbingToTheNeutralPosition);
```

3.16 - ReadZoomInput() and UpdateZoom()

ReadZoomInput() only writes _zoomIsActive. UpdateZoom() only reads that bool to interpolate the FOV. This separation allows activating zoom from an external script without simulating input.

UpdateZoom() also handles the Speed FOV effect when zoom is inactive:

```
        if (_zoomIsActive && Configuration_Configuracion.EnableTheZoomSystem)
        {
            targetFov = Configuration_Configuracion.CameraFieldOfViewDuringZoom;
            transitionSpeed = Configuration_Configuracion.SpeedOfTheFieldOfViewTransitionDuringZoom;
        }
        else
        {
            targetFov = Configuration_Configuracion.BaseFieldOfViewOfTheCamera;
            transitionSpeed = Configuration_Configuracion.SpeedOfTheFieldOfViewTransitionDuringZoom;

            if (Configuration_Configuracion.EnableSpeedFOVEffect)
            {
                float horizontalSpeed = new UnityEngine.Vector2(
                    _playerRigidbody.velocity.x,
                    _playerRigidbody.velocity.z).magnitude;

                float speedThreshold = Configuration_Configuracion.BaseSpeedOfThePlayer *
Configuration_Configuracion.SpeedFOVStartPercent;
                float speedMax       = Configuration_Configuracion.BaseSpeedOfThePlayer *
Configuration_Configuracion.SpeedFOVMaxPercent;

                float speedRatio = Mathf.InverseLerp(speedThreshold, speedMax, horizontalSpeed);
                targetFov = Configuration_Configuracion.BaseFieldOfViewOfTheCamera
                            * Mathf.Lerp(1f, Configuration_Configuracion.SpeedFOVMaxMultiplier, speedRatio);

                transitionSpeed = Configuration_Configuracion.SpeedFOVTransitionSpeed;
            }
        }
```

The interpolated FOV is written to MainCamera_CamaraPrincipal.fieldOfView at the end of Update(), not inside UpdateZoom(). This centralizes FOV writing to a single point per frame.

3.17 - Object Interaction

TryToPickUpTheObject()

```
        Ray ray = new Ray(MainCamera_CamaraPrincipal.transform.position,
MainCamera_CamaraPrincipal.transform.forward);
        if (Physics.Raycast(ray, out RaycastHit hit,
Configuration_Configuracion.MaximumDistanceToPickUpAnObject))
        {
            if (hit.collider.CompareTag(Configuration_Configuracion.TagOfPickableObjects))
                PickUpObject(hit.collider.gameObject);
        }
```

The Raycast has no LayerMask so it detects all colliders. The tag check comes after. This is intentional, using a LayerMask would require the pickable objects to be on a specific layer, whereas the tag approach lets them coexist on any layer.

PickUpObject() and DropObjectInHand()

When picked up, the object's Rigidbody becomes isKinematic = true and velocity = Vector3.zero. When released, it returns to isKinematic = false and inherits the player's current velocity:

```
objectRigidbody.isKinematic = true;
objectRigidbody.velocity = Vector3.zero;
if (DisableTheObjectColliderWhenPickingItUp) collider.enabled = false;


objectRigidbody.isKinematic = false;
objectRigidbody.velocity = _playerRigidbody.velocity;
if (_heldObjectCollider != null) _heldObjectCollider.enabled = true;
```

Without the velocity transfer, the object would fall vertically from its position without inheriting the player's horizontal momentum.

ThrowObjectInHand()

```
 float chargeRatio = _throwInputHoldTime / Configuration_Configuracion.MaximumChargeTimeOfTheObjectThrow;
 float finalForce = Mathf.Lerp(
     Configuration_Configuracion.MinimumObjectThrowForce,
     Configuration_Configuracion.MaximumObjectThrowForce,
     chargeRatio
 );

        if (Physics.Raycast(cameraRay, out RaycastHit hit, 1000f, mask))
            throwDirection = (hit.point - objectToThrow.transform.position).normalized;
        else
            throwDirection = cameraRay.direction;

        DropObjectInHand();

        Rigidbody rb = objectToThrow.GetComponent<Rigidbody>();
        if (rb != null)
        {
            rb.velocity = Vector3.zero;
            if (Configuration_Configuracion.TakeIntoAccountTheMassOfTheObjectWhenThrowingIt)
                rb.AddForce(throwDirection * finalForce, ForceMode.Impulse);
            else
                rb.AddForce(throwDirection * finalForce, ForceMode.VelocityChange);
        }
```

The throw direction is not simply camera.forward. A Raycast is fired from the camera to find the actual aiming point up to 1000 units away (ignoring the held object's own layer). If it hits something, throwDirection = (hit.point - object.position).normalized, so the object travels toward the exact point the crosshair is aimed at. If the ray hits nothing, cameraRay.direction is used as a fallback. ForceMode.VelocityChange: a heavy and a light object launched at the same charge reach the same speed. ForceMode.Impulse: heavier objects travel shorter distances with the same charge.

RotateObjectInHand()

```
        _currentlyHeldObject.transform.Rotate(camTransform.right, -verticalRotationInput * rotationSpeed,
Space.World);
        _currentlyHeldObject.transform.Rotate(camTransform.up, horizontalRotationInput * rotationSpeed,
Space.World);
```

The rotation axes are camTransform.right and camTransform.up, the camera's own axes, not the fixed world axes. Space.World is passed so the rotation is applied in world space using those camera-relative vectors. This means the object tilts up/down relative to where the camera is looking, and rotates left/right relative to the camera's horizontal direction. Space.Self would compound rotations relative to the object's local axes, producing unintuitive behaviour as the object tilts.

3.18 - Slide

StartSliding()

Bypasses ChangeBodyState() and writes directly to _currentBodyState = BodyState.Crouching to avoid the cooldown and posture costs. It also forces the CapsuleCollider height instantly:

```
        _currentBodyState = BodyState.Crouching;
        _timeOfLastBodyStateTransition = Time.time;
        _crouchToggleModeIsActive = false;

        float crouchHeight = Configuration_Configuracion.CapsuleColliderHeightWhileCrouching;
        _playerCapsuleCollider.height = crouchHeight;
        _playerCapsuleCollider.center = new Vector3(0f, crouchHeight * 0.5f, 0f);
```

ProcessArcadeSliding()

```
        _timeSinceSlidingStarted += Time.fixedDeltaTime;

        float progressRatio = _timeSinceSlidingStarted /
Configuration_Configuracion.DurationOfSlidingInArcadeMode;
        float currentSpeed = Mathf.Lerp(
            Configuration_Configuracion.BaseSpeedOfThePlayer *
Configuration_Configuracion.InitialSpeedMultiplierOfArcadeSliding,
            Configuration_Configuracion.MinimumSpeedToKeepSliding,
            progressRatio);

        _playerRigidbody.velocity = new Vector3(
            _slideDirectionAtStart.x * currentSpeed,
            _playerRigidbody.velocity.y,
            _slideDirectionAtStart.z * currentSpeed);
```

The direction is frozen in _slideDirectionAtStart from the first frame. Velocity is assigned directly (not AddForce) to ignore PhysicsMaterial friction and other forces, guaranteeing the deterministic deceleration curve of arcade mode.

ProcessPhysicalSliding()

```
        Vector3 horizontalVelocity = new Vector3(_playerRigidbody.velocity.x, 0f,
_playerRigidbody.velocity.z);
        _playerRigidbody.AddForce(-horizontalVelocity *
Configuration_Configuracion.FrictionDuringPhysicalSliding, ForceMode.Acceleration);

        RaycastHit hit;
        if (Physics.Raycast(transform.position, Vector3.down, out hit,
Configuration_Configuracion.CapsuleColliderHeightWhileStanding,
Configuration_Configuracion.LayersThatAreConsideredGround))
        {
            Vector3 surfaceNormal = hit.normal;
            Vector3 slopeDirection = Vector3.ProjectOnPlane(Vector3.down, surfaceNormal).normalized;
            _playerRigidbody.AddForce(slopeDirection *
Configuration_Configuracion.SlopeAccelerationMultiplierDuringSliding, ForceMode.Acceleration);
        }
```

Vector3.ProjectOnPlane(Vector3.down, normal) calculates the gravity component parallel to the surface plane. On flat ground (normal = Vector3.up), the result is Vector3.zero. On a slope, it points downhill with a magnitude proportional to the angle of inclination.

3.19 - ExecuteDash() - coroutine

```
    private IEnumerator ExecuteDash()
    {
        _playerIsPerformingDash = true;
        _timeSinceLastDash = 0f;

        Vector3 dashDirection = GetMovementDirection();
```

```
        if (dashDirection == Vector3.zero) dashDirection = transform.forward;

        _playerRigidbody.velocity = new Vector3(
            dashDirection.x * Configuration_Configuracion.DashForce,
            _playerRigidbody.velocity.y,
            dashDirection.z * Configuration_Configuracion.DashForce);

        if (Configuration_Configuracion.EnableTheHeadBobbingSystem &&
Configuration_Configuracion.EnableReactiveHeadBobbingWhenDashing)
            _reactiveBobbingImpulse = transform.InverseTransformDirection(dashDirection)
                                      *
Configuration_Configuracion.IntensityOfReactiveHeadBobbingWhenDashing;

        yield return new WaitForSeconds(Configuration_Configuracion.DurationOfTheDashImpulse);

        _playerIsPerformingDash = false;
    }
```

Velocity is overwritten directly to immediately cancel any previous horizontal velocity. Only velocity.y is preserved to avoid interfering with gravity.

transform.InverseTransformDirection(dashDirection) converts the dash direction from World Space to the player's local space. The bob impulse is applied in local space, so the camera tilts in the relative dash direction, forward if the dash is forward, sideways if lateral.

During DurationOfTheDashImpulse seconds, _playerIsPerformingDash = true blocks ApplyMovement(), preventing normal input from interrupting the impulse.

3.20 - Input Layer: DetectGamepadTriggers() and DetectActiveInputDevice()

DetectGamepadTriggers()

Triggers are analog axes, not buttons. To simulate GetButtonDown and GetButtonUp, the current state is compared with the previous frame's state:

```
        _LTWasActiveLastFrame = _LTIsActive;
        _RTWasActiveLastFrame = _RTIsActive;

        _currentLeftTriggerValue = 0f;
        _currentRightTriggerValue = 0f;

        try { _currentLeftTriggerValue = Input.GetAxisRaw("LT"); } catch { }
        try { _currentRightTriggerValue = Input.GetAxisRaw("RT"); } catch { }

        const float triggerThreshold = 0.3f;
        _LTIsActive = _currentLeftTriggerValue > triggerThreshold;
        _RTIsActive = _currentRightTriggerValue > triggerThreshold;

        _LTZoomPressedThisFrame = _LTIsActive && !_LTWasActiveLastFrame;
        _RTReleasedThisFrame = !_RTIsActive && _RTWasActiveLastFrame;
```

The 0.3f threshold avoids accidental activations from analog drift. The _LTZoomPressedThisFrame and _RTReleasedThisFrame signals are true for exactly one frame. On the next tick, when _LTWasActiveLastFrame is updated with the current value, the signals reset automatically.

DetectActiveInputDevice()

Detection uses only JoystickButton KeyCodes (JoystickButton0 to JoystickButton19), never axes. The standard Horizontal and Vertical axes also receive keyboard arrow keys; using them to detect the gamepad would produce false positives. JoystickButton KeyCodes are native to Unity and do not throw exceptions if no gamepad is connected.

If there is input from any JoystickButton - _isUsingGamepad = true. If there is keyboard or mouse input - _isUsingGamepad = false. The change is immediate in the first frame the new device is detected.

4. Public API

Public functions and properties exposed so other project scripts can read or modify the controller state without accessing internal private variables.

> Best practice: to read stamina, use GetNormalizedStamina() instead of PlayerCurrentStamina directly. To modify it, always use ModifyStamina(), it goes through SpendStamina() internally and guarantees _timeElapsedWithoutSpendingStamina is reset.

4.1 - Query Functions

Function	Return	Description
ThePlayerIsOnTheGround()	bool	Returns _playerIsOnGround with the result of the last Physics.CheckSphere in FixedUpdate.
GetNameOfCurrentBodyState()	string	Returns _currentBodyState.ToString(). Values: "Standing", "Crouching", "Prone", "InTheAir".
GetNormalizedStamina()	float	Returns PlayerCurrentStamina / MaximumPlayerStamina. Range: 0.0-1.0.
ThePlayerHasAnObjectInHand()	bool	Returns _currentlyHeldObject != null.
GetTheObjectCurrentlyInHand()	GameObject	Direct reference to _currentlyHeldObject. Null if none.
TheDashIsAvailable()	bool	Returns _timeSinceLastDash >= CooldownBetweenDashUses.
GamepadIsBeingUsed()	bool	Returns _isUsingGamepad, detected in the last Update frame.

4.2 - Modification Functions

Function	Parameters	Description
ModifyStamina(float amount)	float amount	amount < 0: calls SpendStamina(-amount) and resets the regen timer. amount > 0: adds with Mathf.Min, cannot exceed the maximum.
ForceBodyStateChange(string state)	string state	Calls ChangeBodyState() with voluntaryChange = false, bypassing cooldown and costs. Accepts: "standing", "crouching", "prone" (case-insensitive).
ForceDropObjectInHand()	-	Checks if _currentlyHeldObject != null. If so, delegates entirely to DropObjectInHand(), which re-enables isKinematic = false, transfers the player's velocity to the object, and re-enables its collider.

4.3 - Public Variable

Variable	Type	Description
PlayerCurrentStamina	float [HideInInspector]	Stamina in real time. [HideInInspector] hides it from the Inspector. Accessible for direct reading from other scripts.

4.4 - Usage Example

Accessing the controller from an external MonoBehaviour:

```
private PlayerController_ControladorDelJugador _player;

void Start()
{
    _player = FindObjectOfType<PlayerController_ControladorDelJugador>();
}

void Update()
{
    float stamina = _player.GetNormalizedStamina();

    _player.ModifyStamina(-25f);
    _player.ForceBodyStateChange("crouching");

    if (_player.ThePlayerHasAnObjectInHand())
        _player.ForceDropObjectInHand();
}
```